BIGBAZZAR

Software Requirements Specification

AI-Powered, Event-Driven, Multi-Vendor E-Commerce Marketplace

Document purpose: This SRS defines the final BigBazzar project scope, functional requirements, non-functional requirements, system architecture, database design, event-driven workflows, security model, deployment model, and acceptance criteria for an interview-ready three-month implementation.

Version: 1.0
Prepared for: BigBazzar Java SDE Portfolio Project
Date: 17 July 2026

Revision History

Version	Date	Description
1.0	17 July 2026	Initial complete SRS based on final BigBazzar architecture plan.

Table of Contents

1. 1. Introduction
2. 2. Overall Description
3. 3. System Architecture
4. 4. User Classes and Actors
5. 5. Functional Requirements
6. 6. Non-Functional Requirements
7. 7. Microservice Requirements
8. 8. Data and Database Requirements
9. 9. Event-Driven Architecture
10. 10. AI and Recommendation Requirements
11. 11. Security Requirements
12. 12. User Interface Requirements
13. 13. External Interface Requirements
14. 14. Deployment and Operations
15. 15. Testing and Acceptance Criteria
16. 16. Roadmap and Scope Control
17. 17. Appendices

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification is to define the complete requirements for BigBazzar, an AI-powered e-commerce marketplace designed for trust, personalization, transparency, reliable order processing, and scalable system design. The document will guide implementation, testing, deployment, and interview presentation.

1.2 Project Scope

BigBazzar allows customers to discover products, receive AI-assisted recommendations, add products to cart and wishlist, place orders, make payments, track fulfillment, submit verified reviews, and raise support tickets. Sellers can apply, be approved, manage products, manage inventory, view orders, and monitor trust metrics. Admins supervise platform policy, seller approval, categories, risk alerts, audit logs, and exceptional decisions.

1.3 Final Architecture Decision

Final decision: BigBazzar will be implemented as incrementally delivered coarse-grained microservices. Services will communicate through REST for immediate commands and queries, and Kafka for asynchronous facts and workflows. Each service owns its database. The system will be Dockerized and designed for load-balanced deployment.

1.4 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
JWT	JSON Web Token used for stateless authentication
Saga	Distributed workflow coordinated through events and compensating actions
Outbox	Pattern that saves events in the same transaction as business data
Idempotency	Ability to safely retry the same command or event without duplicate side effects
Projection	Local read model built from events owned by another service
Soft delete	Marking records as deleted without physically removing them
SKU	Stock Keeping Unit, usually mapped to a product variant
DLQ	Dead-letter queue or topic for messages that cannot be processed

2. Overall Description

2.1 Product Perspective

BigBazzar is not a simple admin-customer web application. It is a system-driven marketplace where important actions trigger events. The system automatically handles stock reservations, payment outcomes, notifications, search indexing, recommendation signals, audit records, and trust-score recalculation. Humans supervise exceptions and high-risk decisions.

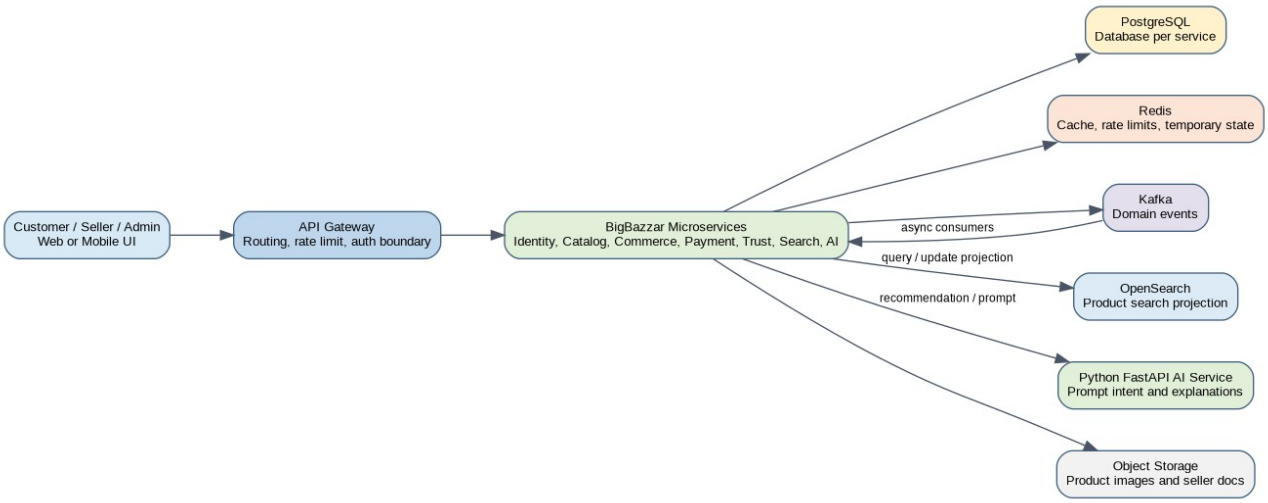


Figure 1 - BigBazzar system context diagram

2.2 Product Vision

Vision: To build a scalable and intelligent e-commerce platform where users can shop through natural-language prompts and receive trustworthy, personalized, explainable recommendations based on real product data.

2.3 Product Differentiators

Capability	BigBazzar behavior
Trust-first shopping	Displays product trust score, seller trust score, verified review signals, and risk indicators.
AI shopping assistant	Converts natural language prompts into structured filters and recommendation explanations.
Price transparency	Shows price history and discount insight instead of only current price.
Inventory reliability	Uses stock reservations and optimistic locking to prevent overselling.
Event-driven automation	Uses Kafka events for notifications, search indexing, analytics, trust, and payment outcomes.
Auditability	Records important actions with actor, entity, reason, old values, new values, and correlation ID.

2.4 Constraints

- The initial implementation must be achievable within approximately three months.
- The system must still represent a realistic Java SDE-level microservices design.
- No service should directly query another service database.
- AI must recommend real catalog products and must not invent product names.
- Payment success must be verified by backend logic, not trusted from frontend-only responses.
- Deployment must be possible with Docker and environment-based configuration.

2.5 Assumptions

- The first release uses sandbox or mock payment integration.
- Object storage can be represented by Cloudinary, S3-compatible storage, or local adapter in development.
- Email can use a sandbox provider in development.
- OpenSearch and Kafka can run locally through Docker Compose during development.

- Advanced ML training and courier integration remain future extensions.

3. System Architecture

3.1 Target Architecture

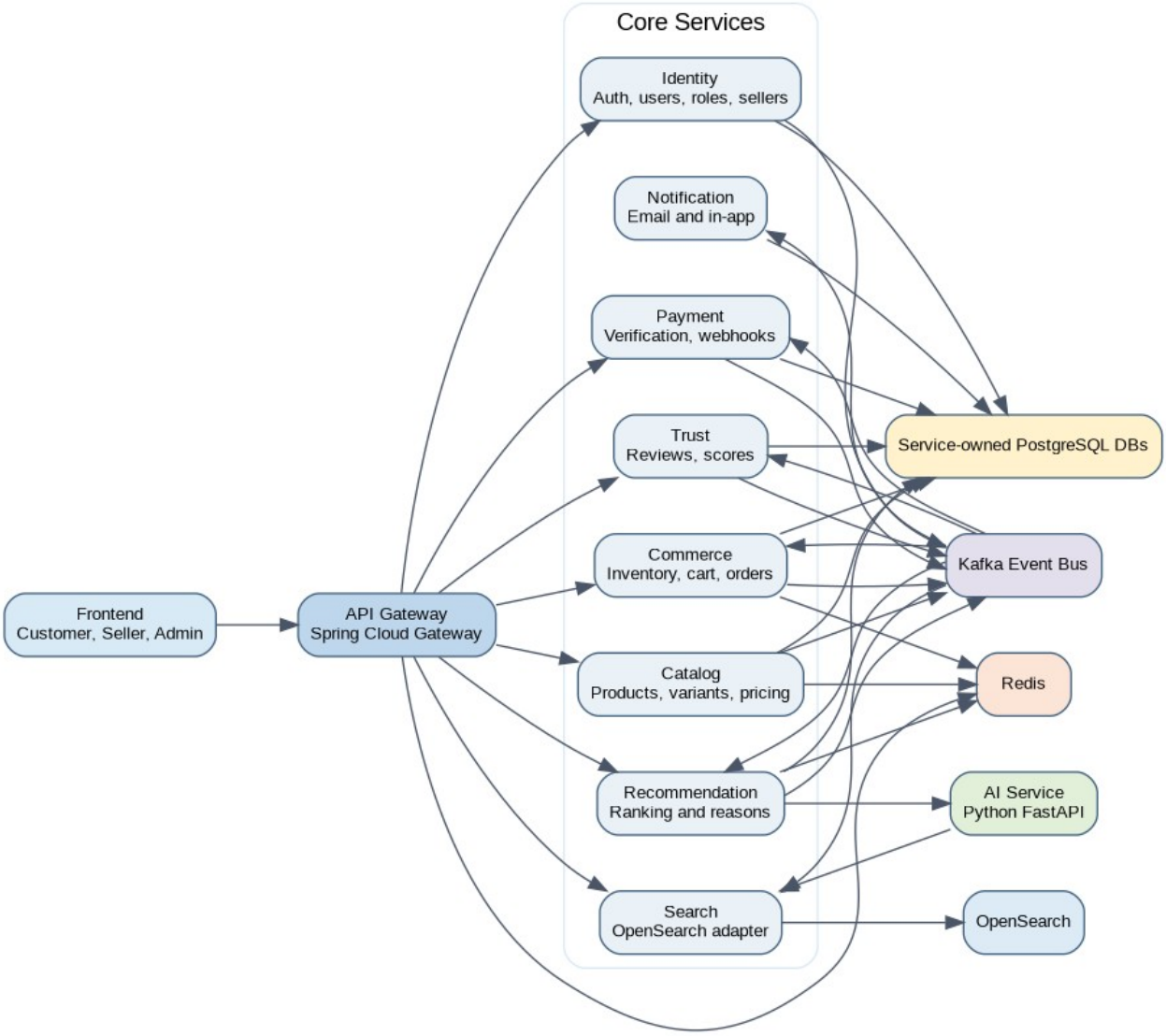


Figure 2 - Target microservice architecture

3.2 Service Build Sequence

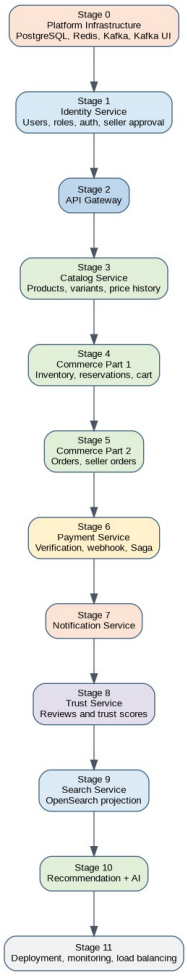


Figure 3 - Incremental service dependency flow

3.3 Architecture Style

The system uses coarse-grained microservices to balance architectural correctness with implementation speed. Identity, Catalog, Commerce, Payment, Notification, Trust, Search, Recommendation, and AI are separated by business responsibility. Closely coupled transactional logic such as inventory and order creation remains in Commerce initially to avoid premature distributed transactions.

3.4 Communication Rules

Communication type	Used for	Examples
REST	Immediate request-response commands and queries	Login, product listing, cart operations, checkout initiation
Kafka events	Business facts and asynchronous reactions	OrderCreated, PaymentSucceeded, ProductPublished, ReviewPublished
Redis	Temporary shared state and cache	Rate limits, product cache, recommendation cache
OpenSearch	Search projection	Keyword search, filters, autocomplete
Object storage	Files and images	Product images, seller documents

3.5 Data Ownership

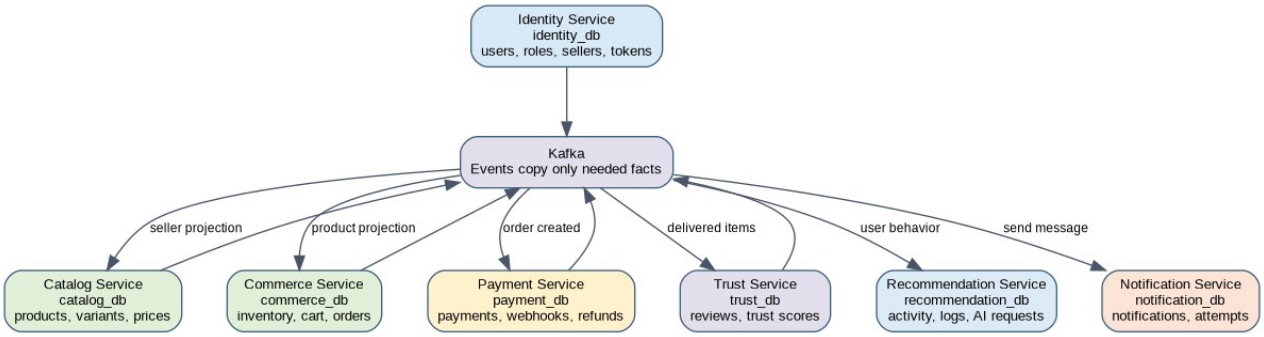


Figure 4 - Database ownership and event-based projections

Rule: A service owns its database. Other services may store logical IDs or projections, but they must not use cross-service foreign keys or direct cross-service table queries.

4. User Classes and Actors

4.1 Human Roles

Role	Capabilities	Restrictions
CUSTOMER	Register, login, browse, search, wishlist, cart, checkout, payment, track orders, review delivered products, raise support tickets, use AI assistant.	Cannot manage products, sellers, platform policy, or another user data.
SELLER	Apply for seller account, manage owned products, images, pricing, inventory, seller orders, performance and trust metrics.	Cannot update another seller product or access customer private data beyond order fulfillment needs.
SUPPORT	View assigned tickets, inspect relevant order/payment context, reply to customers, escalate disputes.	Cannot create admins, approve sellers, change product ownership, or alter payment truth.
ADMIN	Approve/reject sellers, manage categories and risk alerts, review audit logs, suspend accounts, override exceptional cases with reason.	Should not manually control ordinary order workflow; must record reasons for sensitive actions.

4.2 System Actors

Actor	Purpose
SYSTEM	Scheduled jobs, event listeners, automatic stock release, notification triggers, deterministic business rules.
AI_SERVICE	Prompt interpretation, recommendation explanations, review summary, scoring suggestions.

4.3 Role Model Requirements

- Users may hold multiple roles through user_roles.
- Admin-only actions require method-level authorization.
- Seller APIs must verify ownership and approved seller status.
- Suspended users must be blocked from high-risk actions.

- All sensitive role changes must create audit events.

5. Functional Requirements

5.1 Authentication and Identity Requirements

ID	Requirement	Priority
FR-AUTH-01	The system shall allow customers to register with name, email, phone number, and password.	Must
FR-AUTH-02	The system shall hash all passwords using an adaptive hashing algorithm before storage.	Must
FR-AUTH-03	The system shall allow login using email and password and return a JWT access token.	Must
FR-AUTH-04	The system shall support refresh-token rotation and logout/revocation.	Must
FR-AUTH-05	The system shall support multi-role users through user_roles.	Must
FR-AUTH-06	The system shall block suspended or locked users from protected operations.	Must
FR-AUTH-07	The system shall rate-limit login and password-sensitive operations.	Should
FR-AUTH-08	The system shall publish UserRegistered and related identity events.	Must

5.2 Seller Requirements

ID	Requirement	Priority
FR-SELLER-01	A customer shall be able to submit a seller application.	Must
FR-SELLER-02	The admin shall be able to approve, reject, or suspend a seller with a reason.	Must
FR-SELLER-03	The system shall record seller status history as immutable data.	Must
FR-SELLER-04	Only approved sellers shall be able to publish products.	Must
FR-SELLER-05	The seller shall be able to view owned product, inventory, and seller-order data.	Must
FR-SELLER-06	The system shall publish seller lifecycle events.	Must

5.3 Catalog Requirements

ID	Requirement	Priority
FR-CAT-01	The system shall support categories with parent-child hierarchy.	Must
FR-CAT-02	The system shall support brands.	Must
FR-CAT-03	An approved seller shall be able to create products with variants.	Must
FR-CAT-04	Each product variant shall have a unique active SKU.	Must

ID	Requirement	Priority
FR-CAT-05	The system shall support product image metadata and object storage references.	Must
FR-CAT-06	The system shall support product status lifecycle from draft to active, suspended, or deleted.	Must
FR-CAT-07	The system shall store price history when variant prices change.	Must
FR-CAT-08	The system shall publish product and price events.	Must
FR-CAT-09	The system shall support paginated and filtered product listing.	Must

5.4 Inventory and Cart Requirements

ID	Requirement	Priority
FR-INV-01	The seller shall be able to add or adjust stock for owned variants.	Must
FR-INV-02	The system shall maintain on-hand and reserved stock separately.	Must
FR-INV-03	The system shall record stock movements as append-only history.	Must
FR-INV-04	The system shall reserve stock during checkout with expiration.	Must
FR-INV-05	The system shall prevent overselling under concurrent checkout requests.	Must
FR-CART-01	Customers shall be able to add, update, remove, and view cart items.	Must
FR-CART-02	Checkout shall revalidate stock and current price, not trust cart snapshots.	Must
FR-WISH-01	Customers shall be able to manage wishlist items.	Should

5.5 Order Requirements

ID	Requirement	Priority
FR-ORD-01	The system shall create an order from a validated cart.	Must
FR-ORD-02	The system shall split a customer order into seller_orders for multi-vendor fulfillment.	Must
FR-ORD-03	Order items shall store immutable product, SKU, price, and address snapshots.	Must
FR-ORD-04	The system shall maintain order status history.	Must
FR-ORD-05	The system shall support idempotent checkout commands.	Must
FR-ORD-06	The system shall publish OrderCreated and later order lifecycle events.	Must
FR-ORD-07	Customers shall view their own orders; sellers shall view only their seller_orders.	Must

5.6 Payment Requirements

ID	Requirement	Priority
FR-PAY-01	The system shall create a pending payment after an order is created.	Must
FR-PAY-02	The system shall verify payment success on the backend.	Must
FR-PAY-03	The system shall store payment attempts and webhook events.	Must
FR-PAY-04	Duplicate provider webhooks shall be ignored safely.	Must
FR-PAY-05	Payment success shall confirm the order and inventory; payment failure shall release inventory.	Must
FR-PAY-06	The system shall support a payment reconciliation job.	Should
FR-PAY-07	Refund workflow shall be supported in simplified form.	Should

5.7 Review and Trust Requirements

ID	Requirement	Priority
FR-REV-01	Only verified buyers with delivered order items shall be able to review products.	Must
FR-REV-02	The system shall allow one active review per user per delivered order item.	Must
FR-REV-03	The system shall support review moderation states and review flags.	Should
FR-TRUST-01	The system shall calculate product trust score.	Must
FR-TRUST-02	The system shall calculate seller trust score.	Must
FR-TRUST-03	The system shall store trust-score history with reason components.	Must
FR-TRUST-04	The system shall publish trust-score update events.	Must

5.8 Search and Recommendation Requirements

ID	Requirement	Priority
FR-SRCH-01	The system shall index active products into OpenSearch through events.	Must
FR-SRCH-02	The system shall support keyword search, filters, sorting, and pagination.	Must
FR-SRCH-03	The system shall support autocomplete suggestions.	Should
FR-REC-01	The system shall capture product view, search, cart, wishlist, purchase, and review activity.	Must
FR-REC-02	The system shall recommend products using rule-based and content-based ranking.	Must

ID	Requirement	Priority
FR-REC-03	The system shall store recommendation reasons and algorithm version.	Must
FR-AI-01	The AI assistant shall convert prompts into structured product filters.	Must
FR-AI-02	The AI assistant shall recommend only real catalog candidates.	Must
FR-AI-03	The AI assistant shall explain why products were recommended.	Must

5.9 Notification, Support, and Audit Requirements

ID	Requirement	Priority
FR-NOT-01	The notification service shall consume domain events and create notification records.	Must
FR-NOT-02	The system shall support email and in-app notifications.	Must
FR-NOT-03	Notification failure shall not fail order or payment workflows.	Must
FR-SUP-01	Customers shall be able to raise support tickets.	Should
FR-AUD-01	The system shall record important actions as audit events.	Must
FR-AUD-02	Audit events shall contain actor, action, entity, reason, correlation ID, timestamp, and changed values where safe.	Must

6. Non-Functional Requirements

6.1 Performance Requirements

Area	Target
Cached product detail	Less than 300 ms under normal development-load conditions
Search query	Less than 700 ms for common filters in MVP environment
Cart update	Less than 300 ms
Order creation before payment	Less than 2 seconds excluding payment provider
AI recommendation	Less than 5 seconds with fallback if AI is unavailable
Notification processing	Asynchronous and eventually processed with retry

6.2 Scalability Requirements

- Each service shall be stateless and horizontally scalable.
- The gateway and services shall support multiple replicas behind a load balancer.
- High-read services such as Catalog, Search, and Recommendation shall be independently scalable.
- Kafka consumers shall use consumer groups for scalable event processing.
- Redis and OpenSearch shall be external shared infrastructure, not embedded in application memory.

6.3 Availability Requirements

- Product browsing shall remain available if the AI service is unavailable.
- Payment and order truth shall remain consistent even if notification fails.
- OpenSearch failure shall not corrupt catalog data; indexing shall retry or be reconciled.
- Each service shall expose health/readiness endpoints.

6.4 Reliability Requirements

- Critical commands shall be idempotent.
- Kafka consumers shall be idempotent.
- Payment webhooks shall be deduplicated.
- Outbox events shall use retry and dead-letter handling.
- Scheduled jobs shall be idempotent or protected by distributed locks.
- Reconciliation jobs shall detect and repair stuck payment/order states.

6.5 Maintainability Requirements

- Each service shall have a clear package structure, README, OpenAPI documentation, and Flyway migrations.
- Controllers shall not contain business logic.
- Entities shall not be exposed as API responses.
- Common code sharing shall be minimal and shall not include shared JPA entities.
- Architecture Decision Records shall document major trade-offs.

6.6 Observability Requirements

- All logs shall include service name and correlation ID.
- Kafka events shall include event ID, event type, version, source service, and correlation ID.
- Metrics shall include latency, error rate, request count, cache hit ratio, Kafka lag, payment failures, and inventory reservation conflicts.
- The system shall support health checks, structured logs, and dashboard-ready metrics.

7. Microservice Requirements

Service	Stack	Data owner	Responsibilities	Events
Identity	Java Spring Boot	identity_db	Auth, users, roles, sellers, seller approval, tokens	UserRegistered, SellerApproved
API Gateway	Spring Cloud Gateway	None	Routing, CORS, rate limiting, correlation IDs, auth boundary	None
Catalog	Java Spring Boot	catalog_db	Categories, brands, products, variants, images, price history	ProductPublished, ProductPriceChanged
Commerce	Java Spring Boot	commerce_db	Inventory, reservations, cart, wishlist, orders, seller_orders	OrderCreated, StockReserved
Payment	Java Spring Boot	payment_db	Payment attempts, verification, webhooks, refunds	PaymentSucceeded, PaymentFailed
Notification	Java Spring Boot	notification_db	Email, in-app messages, retries, delivery history	Consumes events
Trust	Java Spring Boot	trust_db	Verified reviews, flags, trust scores, score history	ReviewPublished, TrustScoreUpdated
Search	Java Spring Boot + OpenSearch	OpenSearch	Indexing, search, filters, autocomplete	Consumes product/stock/trust events

Service	Stack	Data owner	Responsibilities	Events
Recommendation	Java or Python/Java mix	recommendation_db	User activity, ranking, reasons, logs	RecommendationGenerated
AI Service	Python FastAPI	Optional ai logs	Intent extraction, comparison, review summary, explanations	None or AIRequestLogged

7.1 Service Completion Gate

18. Flyway migrations run on an empty database.
19. Unit tests and integration tests pass.
20. Security and ownership tests pass where applicable.
21. Kafka producer or consumer tests pass where applicable.
22. Docker image builds and health endpoint works.
23. OpenAPI/Postman examples are updated.
24. Service README explains responsibilities and failure handling.

8. Data and Database Requirements

8.1 Database Strategy

The system uses PostgreSQL database-per-service ownership. In local development, one PostgreSQL server may host multiple databases. In production, each service must have its own credentials and cannot directly access another service database.

8.2 Common Data Conventions

Concern	Requirement
IDs	Use UUID for public/business entities. Use BIGINT for high-volume append-only logs where useful.
Time	Use TIMESTAMPTZ in PostgreSQL and Instant in Java.
Money	Use NUMERIC(12,2) in PostgreSQL and BigDecimal in Java.
Enums	Store string enum values; never use ordinal values.
Schema	Use Flyway migrations. Use Hibernate ddl-auto=validate in deployable environments.
Deletes	Use selective soft delete and immutable history where required.
Indexes	Index based on access patterns and avoid unnecessary write-heavy indexes.
Pagination	All large listings must be paginated.

8.3 Soft Delete Requirements

Soft-deletable entities shall use deleted_at, deleted_by, and delete_reason. A record is active when deleted_at is null. Transactional history such as orders, payments, stock movements, price history, order status history, and audit events shall not be soft-deleted in normal operation.

8.4 Core Database Modules

Database	Important tables
identity_db	users, roles, user_roles, addresses, sellers, seller_documents, seller_status_history, refresh_tokens, outbox_events
catalog_db	approved_seller_projection, categories, brands, products,

Database	Important tables
	product_variants, product_images, price_history, outbox_events, processed_events
commerce_db	product_projection, inventory, stock_reservations, stock_movements, carts, cart_items, wishlists, orders, seller_orders, order_items, order_status_history, idempotency_keys, outbox_events, processed_events
payment_db	payments, payment_attempts, payment_webhook_events, refunds, outbox_events, processed_events
notification_db	notifications, delivery_attempts, preferences, processed_events
trust_db	delivered_item_projection, reviews, review_flags, review_summaries, product_scores, seller_scores, score_history, outbox_events, processed_events
recommendation_db	user_activity, user_preferences, recommendation_logs, recommendation_items, product_similarity, ai_requests, processed_events
platform/audit	audit_events, support_tickets, ticket_messages

8.5 Important Index and Constraint Requirements

```
CREATE UNIQUE INDEX uk_users_email_active
ON users (LOWER(email))
WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX uk_product_variants_sku_active
ON product_variants (sku)
WHERE deleted_at IS NULL;

ALTER TABLE inventory
ADD CONSTRAINT chk_inventory_available
CHECK (on_hand_quantity >= reserved_quantity);

CREATE INDEX idx_orders_user_created
ON orders (user_id, created_at DESC);

CREATE INDEX idx_reservations_expiry
ON stock_reservations (status, expires_at)
WHERE status = 'ACTIVE';
```

8.6 Inventory Data Requirements

Inventory formula: available_quantity = on_hand_quantity - reserved_quantity. Reservation succeeds only when available_quantity is greater than or equal to requested quantity.

8.7 Order Data Requirements

- Orders must store address snapshots.
- Order items must store product, SKU, variant, and price snapshots.
- Orders must be split into seller_orders for multi-vendor fulfillment.
- Order status changes must be stored in append-only order_status_history.
- Checkout must use idempotency keys.

9. Event-Driven Architecture

9.1 Event Standards

```
public record EventEnvelope<T>(
    UUID eventId,
    String eventType,
    int eventVersion,
    String sourceService,
    UUID correlationId,
    Instant occurredAt,
    T payload
) {}
```

9.2 Transactional Outbox Pattern



Figure 5 - Transactional outbox and idempotent consumer pattern

9.3 Event Catalog

Topic / Area	Events
identity-events	UserRegistered, UserVerified, SellerApplicationSubmitted, SellerApproved, SellerRejected, UserSuspended
product-events	ProductCreated, ProductUpdated, ProductPublished, ProductDeactivated, ProductPriceChanged
commerce-events	StockReserved, StockReleased, OrderCreated, OrderConfirmed, OrderCancelled, OrderDelivered
payment-events	PaymentInitiated, PaymentSucceeded, PaymentFailed, RefundRequested, RefundCompleted
trust-events	ReviewSubmitted, ReviewPublished, ReviewFlagged, ProductTrustScoreUpdated, SellerTrustScoreUpdated
recommendation-events	ProductViewed, ProductWishlisted, ProductAddedToCart, ProductPurchased, RecommendationClicked
notification-events	NotificationCreated, NotificationSent, NotificationFailed

9.4 Consumer Requirements

- Every consumer shall record processed event IDs with consumer name.
- Consumers shall ignore duplicate events safely.
- Consumers shall use retry and dead-letter handling for failures.
- Consumers shall log event ID, correlation ID, and failure reason.
- Event schemas shall be versioned and backward-compatible when possible.

10. Critical Workflow Requirements

10.1 Checkout and Payment Saga

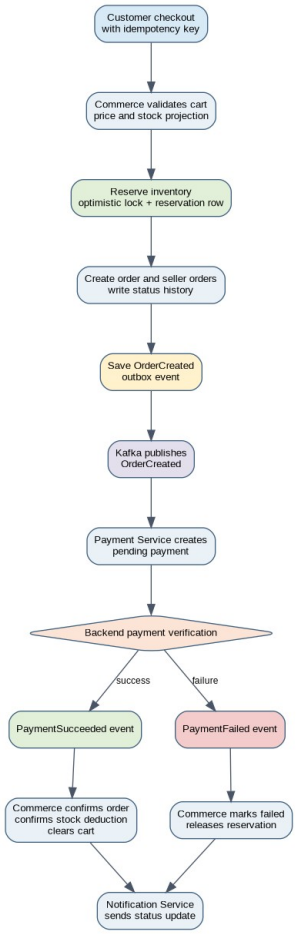


Figure 6 - Checkout and payment Saga

10.2 Product Publication Flow

Approved seller creates product
-> Catalog validates seller projection
-> Catalog stores product and variants
-> Catalog writes ProductPublished to outbox
-> Kafka publishes event
-> Commerce updates product projection
-> Search indexes product
-> Audit records action

10.3 Review and Trust Flow

OrderDelivered event
-> Trust stores delivered item projection
-> Customer submits review
-> Trust verifies order item eligibility
-> ReviewPublished event
-> Product and seller trust recalculated
-> Catalog/Search/Recommendation receive trust updates

10.4 Payment Failure Flow

```
PaymentFailed event
  -> Commerce marks order payment failed
  -> Commerce releases active reservation
  -> Stock movement is appended
  -> Notification informs customer
  -> Analytics records failure
```

11. AI and Recommendation Requirements

11.1 AI Assistant Requirements

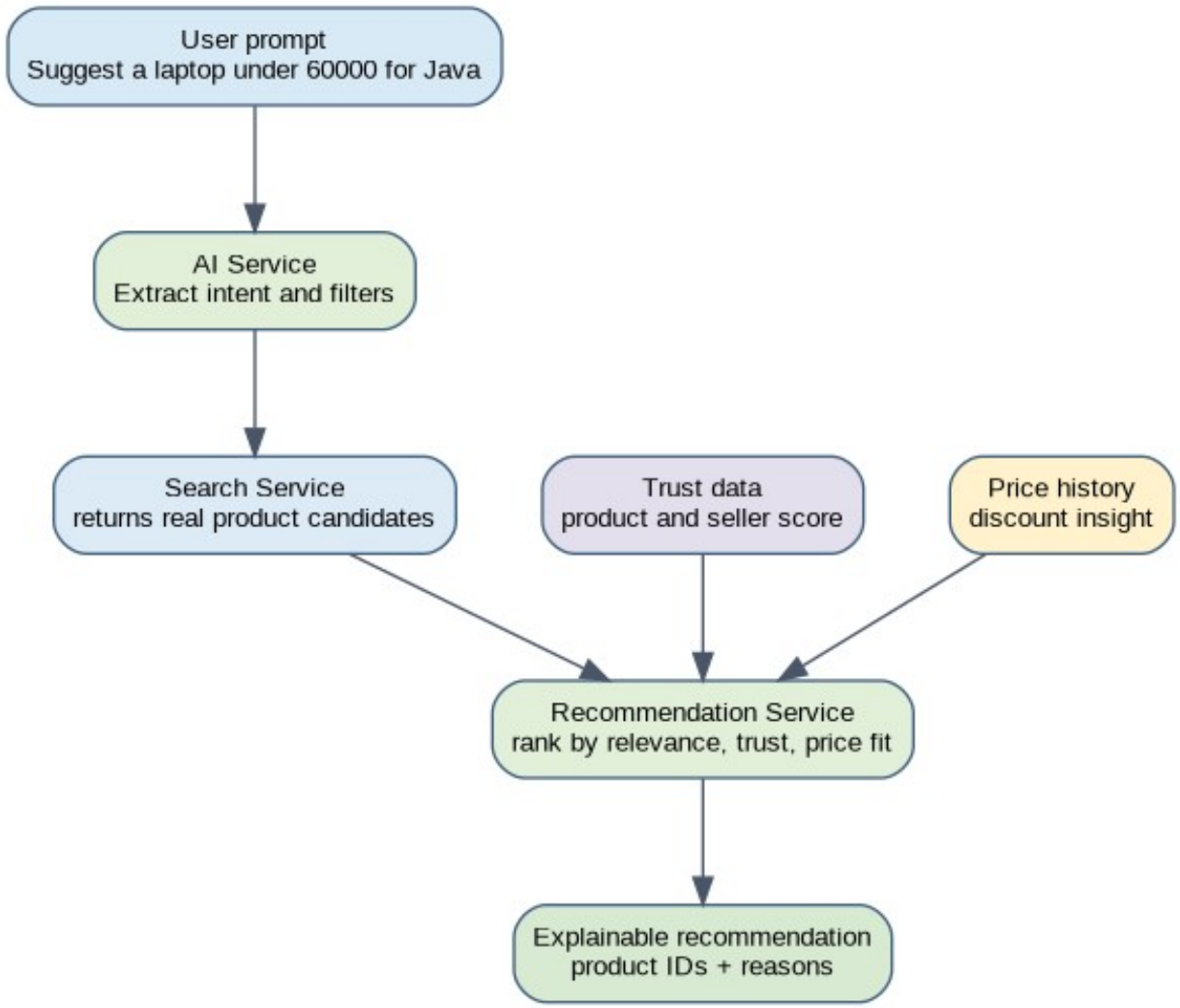


Figure 7 - AI prompt-based recommendation flow

- The AI service shall extract category, budget, preferences, and use case from natural language prompts.
- The AI service shall return structured filters, not invented products.
- The Search service shall provide real catalog candidates.
- The Recommendation service shall rank candidates using relevance, trust, price fit, rating, seller score, review quality, and availability.
- The final response shall include product IDs and explanations.

- If AI fails, the system shall fall back to rule-based recommendations.

11.2 Ranking Formula - Version 1

```
Final Score =
0.25 * Relevance Score
+ 0.20 * Product Trust Score
+ 0.15 * Rating Score
+ 0.15 * Price Fit Score
+ 0.10 * Seller Score
+ 0.10 * Review Quality Score
+ 0.05 * Delivery / Stock Confidence
```

11.3 Recommendation Logs

The system shall store recommendation logs, recommendation items, ranks, scores, reason codes, algorithm version, clicks, cart additions, and purchases. These logs will support explainability and future model improvement.

12. Security Requirements

Area	Requirement
Authentication	JWT access token, refresh-token rotation, logout/revocation, BCrypt password hashing.
Authorization	Role-based access and ownership checks for seller resources.
API protection	Rate limiting, validation, error sanitization, CORS configuration.
Payment	Backend verification, signed webhook validation, duplicate event detection.
Secrets	No secrets in source code; use environment variables locally and secret manager in deployment.
Audit	Sensitive admin actions require reason and audit record.
Data protection	Avoid logging tokens, passwords, payment secrets, or sensitive documents.
File uploads	Validate type, size, and storage path; store files outside PostgreSQL.

12.1 Standard Error Response

```
{
  "success": false,
  "code": "INSUFFICIENT_STOCK",
  "message": "Requested quantity is unavailable.",
  "correlationId": "uuid",
  "timestamp": "2026-07-17T10:30:00Z",
  "fieldErrors": []
}
```

13. User Interface Requirements

13.1 Public UI

- Home page with search and recommendation sections.
- Product listing with filters, sort, pagination, trust and price information.
- Product detail page with variants, trust score, seller score, price history, stock state, reviews, and AI summary.
- Login and registration pages.

13.2 Customer UI

- Profile and address management.
- Cart and wishlist pages.
- Checkout and payment page.
- Order history and tracking.
- Verified review submission.
- AI shopping assistant and recommendation history.
- Support ticket page.

13.3 Seller UI

- Seller application and status page.
- Product management dashboard.
- Inventory management.
- Seller order queue.
- Price and trust performance insights.

13.4 Admin UI

- Seller approval and suspension.
- Category and brand management.
- Risk and review flags.
- Audit logs.
- Platform metrics and system health.

14. Deployment and Operations Requirements

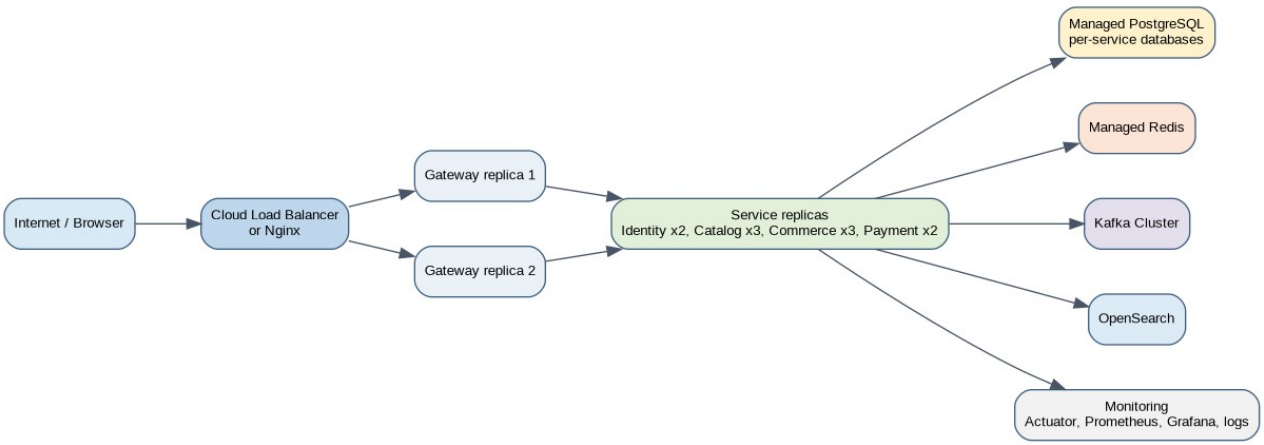


Figure 8 - Deployment and load-balancing architecture

14.1 Environments

Environment	Purpose
Local development	Docker Compose for PostgreSQL, Redis, Kafka, OpenSearch, and local services.
Integration	All completed services run through gateway with test data.
Portfolio deployment	Cloud/container deployment with managed database, Redis, object storage, and HTTPS.
Future production	Managed Kafka/OpenSearch and independent autoscaling per service.

14.2 Load Balancing Requirements

- Gateway and service instances shall be stateless.
- Multiple replicas shall be supported for gateway and services.
- Load balancer shall route only to healthy replicas.
- Sessions shall not be stored in JVM memory.
- Shared state shall live in PostgreSQL, Redis, Kafka, OpenSearch, or object storage.

14.3 CI/CD Requirements

- Build and test every service independently.
- Run migrations in controlled deployment order.
- Build Docker images.
- Run integration tests before deployment.
- Use environment-specific configuration.

15. Testing and Acceptance Criteria

15.1 Test Types

Test type	Coverage
Unit tests	Business rules, mappers, score formulas, validators, status transitions.
Repository tests	Constraints, soft delete, locking, and queries with Testcontainers.
Controller tests	API validation, auth, error responses, pagination.
Security tests	Role access, seller ownership, suspended-account rejection.
Kafka tests	Producer, consumer, idempotency, retry, DLQ behavior.
Concurrency tests	Two users attempting to reserve the final unit.
End-to-end tests	Register -> seller approve -> product -> cart -> order -> payment -> review -> trust.
Resilience tests	AI unavailable, notification provider unavailable, duplicate webhook, duplicate Kafka event.

15.2 Acceptance Criteria

25. Users can register, log in, refresh tokens, and access role-appropriate APIs.
26. Admins can approve sellers and the approval is auditable.
27. Approved sellers can publish products with variants and inventory.
28. Customers can search, add to cart, checkout, and place orders.
29. Concurrent checkout cannot oversell inventory.
30. Payment success confirms order and stock; payment failure releases stock.
31. Notifications are processed asynchronously.
32. Only delivered order items can be reviewed.
33. Trust scores update and are visible in product discovery.
34. AI recommendations return real product candidates and explain reasons.
35. Services run through Docker and the gateway.
36. Core diagrams, API docs, tests, and README files are complete.

16. Roadmap and Scope Control

16.1 MVP Must Have

- Identity, authentication, roles, seller approval.
- Catalog, products, variants, price history.
- Inventory reservation, cart, order, seller_orders.
- Payment sandbox, backend verification, Saga events.
- Notifications, verified reviews, trust scores.
- Search, recommendation, AI assistant.
- Docker deployment, monitoring, tests and documentation.

16.2 Simplify Initially

- Use sandbox payment provider.
- Use deterministic trust score before advanced ML.
- Use email and in-app notification before SMS/push.
- Use rule-based/content-based recommendation before complex collaborative filtering.
- Use Docker/cloud container deployment before Kubernetes.

16.3 Future Enhancements

- Real courier integration and live delivery tracking.
- Advanced fraud and fake-review ML.
- Seller payout automation.
- Complex refund automation.
- Multi-region deployment.
- Kubernetes and predictive autoscaling.
- Full warehouse management.

17. Appendices

Appendix A - Status Enums

UserStatus: PENDING_VERIFICATION, ACTIVE, LOCKED, SUSPENDED, DEACTIVATED
SellerStatus: DRAFT, PENDING_VERIFICATION, UNDER_REVIEW, APPROVED, REJECTED, SUSPENDED, CLOSED
ProductStatus: DRAFT, PENDING_APPROVAL, ACTIVE, INACTIVE, OUT_OF_STOCK, REJECTED, SUSPENDED, DELETED
ReservationStatus: ACTIVE, CONFIRMED, RELEASED, EXPIRED, CANCELLED
OrderStatus: CREATED, INVENTORY_RESERVED, PAYMENT_PENDING, PAYMENT_SUCCESS, PAYMENT_FAILED, CONFIRMED, PACKED, SHIPPED, DELIVERED, CANCELLED, REFUNDED
PaymentStatus: CREATED, PENDING, AUTHORIZED, CAPTURED, FAILED, CANCELLED, PARTIALLY_REFUNDED, REFUNDED
ReviewStatus: PUBLISHED, PENDING_MODERATION, FLAGGED, HIDDEN, REMOVED

Appendix B - Interview Focus Points

- Why BigBazzar uses coarse-grained microservices instead of twenty services at once.
- How inventory reservation prevents overselling.
- Why outbox and idempotency are necessary with Kafka.
- How payment Saga handles success, failure, and retries.
- Why OpenSearch is a projection and PostgreSQL remains source of truth.
- How soft delete differs from immutable transactional history.
- How AI recommendations are grounded in real catalog data.
- How load balancing works with stateless services.

Appendix C - Implementation Timeline Summary

Phase	Weeks	Outcome
Platform + Identity + Gateway	1-2	Authentication, roles, seller onboarding, gateway routing.
Catalog + Commerce Foundation	3-5	Products, inventory, cart, orders, seller orders.
Payment + Notification + Trust	6-8	Payment Saga, asynchronous notification, verified reviews, trust scores.
Search + AI + Deployment	9-12	OpenSearch, recommendations, AI assistant, frontend integration, monitoring, deployment.